Learn ESP32 for Home Automation

Lesson 2: Understanding GPIO, PWM and Digital I/O

Understanding GPIO, PWM and Digital I/O – Study Notes

Key Concepts

1. **GPIO (General Purpose Input/Output)** – configurable pins that can act as digital inputs or outputs.
2. **Digital I/O** – reading a binary state (HIGH/LOW) from a sensor or driving a binary load (LED, relay, etc.).
3. **PWM (Pulse Width Modulation)** – a technique to simulate an analog voltage by rapidly switching a digital pin on and off with a controllable duty cycle.
4. **Debouncing** – eliminating false transitions when reading mechanical push buttons.
5. **Hardware considerations** – pull up/pull down resistors, current limits, and safe LED driving.

Summary

GPIO pins are the bridge between a microcontroller (e.g., Arduino, Raspberry Pi, ESP32) and the outside world. By configuring a pin as **output**, you can drive devices such as LEDs, buzzers, or relays. When configured as **input**, the same pin can sense the state of switches, sensors, or other digital signals.

Digital I/O works with two discrete voltage levels: **HIGH** (typically 3.3 /V or 5 /V) and **LOW** (0 /V). For reliable button reads you must manage **debouncing**—either in hardware (RC filter, Schmitt trigger) or in software (delay or state machine).

PWM extends the capability of a digital output by rapidly toggling the pin between HIGH and LOW. The proportion of the “on” time within each cycle is the **duty cycle** (0 /%–100 /%). By varying the duty cycle you can dim LEDs, control motor speed, or generate audio tones, all while keeping the pin in digital mode. Most microcontrollers provide hardware PWM channels that free the CPU from timing the toggles manually.

Important Points

- **Pin Modes**
- `pinMode(pin, OUTPUT);` – set as digital output.
- `pinMode(pin, INPUT);` – high impedance input (requires external pull up/down).
- `pinMode(pin, INPUT_PULLUP);` – enable internal pull up resistor (Arduino).
- **Reading a Button**
- Use a pull up or pull down resistor so the input is never floating.
- Typical wiring: button to ground + internal pull up !’ `LOW` when pressed.
- **Debouncing Techniques**
- **Hardware:** 10–100 /k:’ &W6—7F÷" ² ã ûTb 6 6—F÷" …$2 Æðw pass).

- **Software:** `delay(10);` after a state change, or use a non blocking timer (`millis()`).

• **Driving an LED**

- Calculate resistor: `R = (Vcc – Vf) / I` (e.g., 5 /V – 2 /V) / 20 /mA "H 150 /:'à
- Do **not** exceed the MCU's per pin current rating (typically 20 /mA).

• **PWM Basics**

- Frequency: number of cycles per second (e.g., 490 /Hz on Arduino Uno pins 5 & 6, 980 /Hz on pins 3, 9, 10, 11).
- Duty cycle: `analogWrite(pin, value);` where `value` is 0–255 (0 /%–100 /%).
- For smoother dimming, use higher resolution PWM (e.g., ESP32's 12 bit PWM).

• **Safety**

- Never connect a pin directly to a voltage higher than its VCC.
- Use series resistors for LEDs and transistors or MOSFETs for loads > /20 /mA.

Examples

1þ ã Blink an LED & Read a Push Button (Arduino)

```cpp
// Pin definitions
const int ledPin   = 13;   // built  in LED
const int btnPin   = 2;    // push  button connected to GND
// Variables for debouncing
bool lastState = HIGH;
unsigned long lastDebounce = 0;
const unsigned long debounceDelay = 50; // ms
void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(btnPin, INPUT_PULLUP); // enable internal pull  up
}
void loop() {
  // ----- Read button with software debounce -----
  bool reading = digitalRead(btnPin);
  if (reading != lastState) {
    lastDebounce = millis();          // reset timer
  }
  if ((millis() - lastDebounce) > debounceDelay) {
    // stable state reached
    if (reading != lastState) {
      lastState = reading;
      if (reading == LOW) {           // button pressed
        digitalWrite(ledPin, !digitalRead(ledPin)); // toggle LED
      }
    }
```

```
  }
}
```

Explanation: The button is wired to ground; the internal pull up holds the pin HIGH when released. When pressed, the pin reads LOW, toggling the LED. Debouncing prevents multiple toggles from a single press.

2þ ã Dim an LED with PWM (Arduino)

```cpp
const int pwmPin = 9;          // PWM  capable pin
int brightness = 0;            // 0  255
int fadeAmount = 5;            // step size
void setup() {
  pinMode(pwmPin, OUTPUT);
}
void loop() {
  analogWrite(pwmPin, brightness); // set duty cycle
  brightness += fadeAmount;        // change brightness
  // Reverse direction at limits
  if (brightness <= 0 || brightness >= 255) {
    fadeAmount = -fadeAmount;
  }
  delay(30); // visible fade speed
}
```

Explanation: `analogWrite()` sets the duty cycle. The code ramps the duty cycle up and down, producing a smooth fade. The PWM frequency is fixed by the MCU ("H490 /Hz on pin /9), which is fast enough that the eye perceives a continuous change in brightness.

Quick Review

1. **What is the purpose of a pull up resistor when reading a mechanical switch?**
2. **How does PWM create the illusion of an analog voltage?**
3. **Why must you limit the current through a GPIO pin, and how do you calculate the required series resistor for an LED?**
4. **Describe one software method for debouncing a button.**
5. **If you need a PWM frequency higher than the default, what options do you have on most microcontrollers?**

Further Reading

- **Interrupt driven button handling** – using `attachInterrupt()` for responsive input.
- **Hardware PWM vs. software PWM** – advantages, limitations, and when to use each.

- **Using transistors/MOSFETs as switches** – driving motors, relays, or high current LEDs.
- **Analog vs. PWM based dimming** – perceived brightness and gamma correction.
- **Microcontroller specific PWM libraries** – e.g., `ESP32LEDC`, `TimerOne` for Arduino.

End of study notes.